

----> JOINS

1. Title

Replace Employee ID With the Unique Identifier

What to Find

- Display **each employee's name**
- Show their **unique_id** if it exists
- If an employee **does not have a unique_id**, display **NULL**
- Result can be returned **in any order**

Tables

Employees

Column	Type
id	int
name	varchar

EmployeeUNI

Column	Type
id	int
unique_id	int

Hint

- Use a **LEFT JOIN**
- Start from the **Employees** table
- This ensures **all employees appear**
- Missing matches will automatically show **NULL**

Solution

```
SELECT eu.unique_id, e.name
FROM Employees e
LEFT JOIN EmployeeUNI eu
ON e.id = eu.id;
```

2. Title

Product Sales Analysis

What to Find

- For **each sale** in the `Sales` table:
 - Find the **product_name**
 - Show the **year**
 - Show the **price**
- Return the result in **any order**

Tables

Sales

Column	Description
<code>sale_id</code>	Unique sale identifier
<code>product_id</code>	Product sold
<code>year</code>	Year of sale
<code>quantity</code>	Units sold
<code>price</code>	Price per unit

Product

Column	Description
<code>product_id</code>	Unique product identifier
<code>product_name</code>	Name of product

Solution (MySQL)

```
SELECT p.product_name, s.year, s.price
FROM Sales s
JOIN Product p
ON s.product_id = p.product_id;
```

3. Title

Customer Who Visited but Did Not Make Any Transactions

What to Find

- Identify **customers who visited the mall but made no transactions**
- Count **how many such visits** each customer made
- Return:
 - `customer_id`
 - `count_no_trans` → number of visits without transactions
- Order of result **does not matter**

Tables

Visits

Column	Description
<code>visit_id</code>	Unique visit identifier
<code>customer_id</code>	Customer who visited

Transactions

Column	Description
<code>transaction_id</code>	Unique transaction
<code>visit_id</code>	Visit during which transaction occurred
<code>amount</code>	Transaction amount

Hint

- Use a **LEFT JOIN** from Visits to Transactions
- Visits with **no matching transaction** will have `NULL`
- Filter using `WHERE t.visit_id IS NULL`
- Group by `customer_id` and count visits

Solution

```
SELECT
    v.customer_id,
    COUNT(v.visit_id) AS count_no_trans
FROM Visits v
LEFT JOIN Transactions t
ON v.visit_id = t.visit_id
WHERE t.visit_id IS NULL
GROUP BY v.customer_id;
```